Bhartiya Vidya Bhavan's, Sardar Patel Institute of Technology
(An Autonomous Institute Affiliated to University of Mumbai)
Bhavan's Campus, Munshi Nagar, Andheri (West),
Mumbai-400058-India

Academic Year: 2025-26 **Class: TE Comp** **Div: C**
Subject: SPCC **Sem: 6**

Name	Bhavik Dandekar
UID no.	2023300036
Aim:	Exp 7 : Code Generation

Theory:	Basic Blocks and Flow Graph In compiler design, the analysis of a program's control flow is essential for optimization and efficient code generation. Two fundamental concepts used for this purpose are Basic Blocks and Flow Graphs. 🔹 Basic Block A Basic Block is a sequence of consecutive three-address statements with the following properties: ● It has only one entry point (the first statement). ● It has only one exit point (the last statement). ● Control enters at the beginning and exits at the end without any jumps or branches inside the block. Thus, once execution enters a basic block, all statements within it are executed sequentially. 🔹 Leaders To divide a program into basic blocks, we first identify leaders. A leader is the first statement of a basic block. The rules for identifying leaders are: 1. The first statement of the program is always a leader. 2. Any statement that is the target of a conditional or unconditional goto is a leader.

3. Any statement that immediately follows a goto or conditional goto is also a leader.

🔹 Formation of Basic Blocks

Once leaders are identified:

- Each leader forms the start of a basic block.
- A basic block includes the leader and all subsequent statements up to (but not including) the next leader or the end of the program.

🔹 Flow Graph (Control Flow Graph)

A Flow Graph is a directed graph that represents the flow of control between basic blocks.

Components:

- Nodes → Represent basic blocks
- Edges → Represent possible flow of control

🔹 Types of Control Flow

1. Sequential Flow
   - Control moves from one block to the next in sequence.
   - Occurs when there is no jump instruction.
2. Conditional Jump (if–goto)
   - Creates two outgoing edges:
     - One to the target block
     - One to the next sequential block (fall-through)
3. Unconditional Jump (goto)
   - Creates only one edge:
     - From current block to the target block
   - No fall-through occurs.

🔹 Construction of Flow Graph

To construct the flow graph:

1. Identify all basic blocks.
2. Represent each block as a node.
3. Add directed edges:
   - From a block containing a conditional jump to both:

- Target block
- Next block

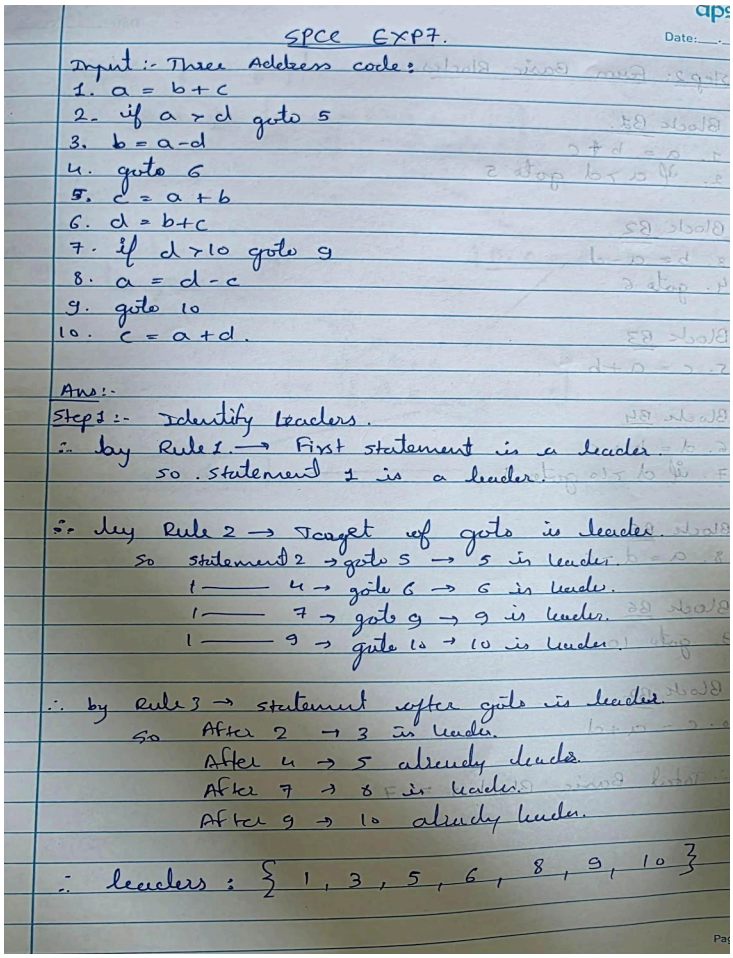
  - From a block containing an unconditional jump to its target block
  - From a block without jumps to the next sequential block

🔹 Applications

Basic blocks and flow graphs are used in:

- Code optimization (dead code elimination, loop optimization)
- Data flow analysis
- Register allocation
- Detecting unreachable code
- Improving execution efficiency

Manually Solved Problem



SPCE EXP7.

Input :- Three Address code :

1. a = b + c
2. if a > d goto 5
3. b = a - d
4. goto 6
5. c = a + b
6. d = b + c
7. if d > 10 goto 9
8. a = d - c
9. goto 10
10. c = a + d.

Ans :-

Step 1 :- Identify leaders.

∴ by Rule 1. → First statement is a leader.
so. statement 1 is a leader.

∴ by Rule 2 → Target of goto is leader.
So statement 2 → goto 5 → 5 is leader.
1 —— 4 → goto 6 → 6 is leader.
1 —— 7 → goto 9 → 9 is leader.
1 —— 9 → goto 10 → 10 is leader.

∴ by Rule 3 → statement after goto is leader.
So After 2 → 3 is leader.
After 4 → 5 already leader.
After 7 → 8 is leader.
After 9 → 10 already leader.

∴ leaders : { 1, 3, 5, 6, 8, 9, 10 }

Step 2: Form Basic Blocks.

1) Block B1.
1. a = b + c
2. if a > d goto 5

2) Block B2
3. b = a - d
4. goto 6

3) Block B3
5. c = a + b

4) Block B4
6. d = b + c
7. if d > 10 goto 9

5) Block B5
8. a = d - c

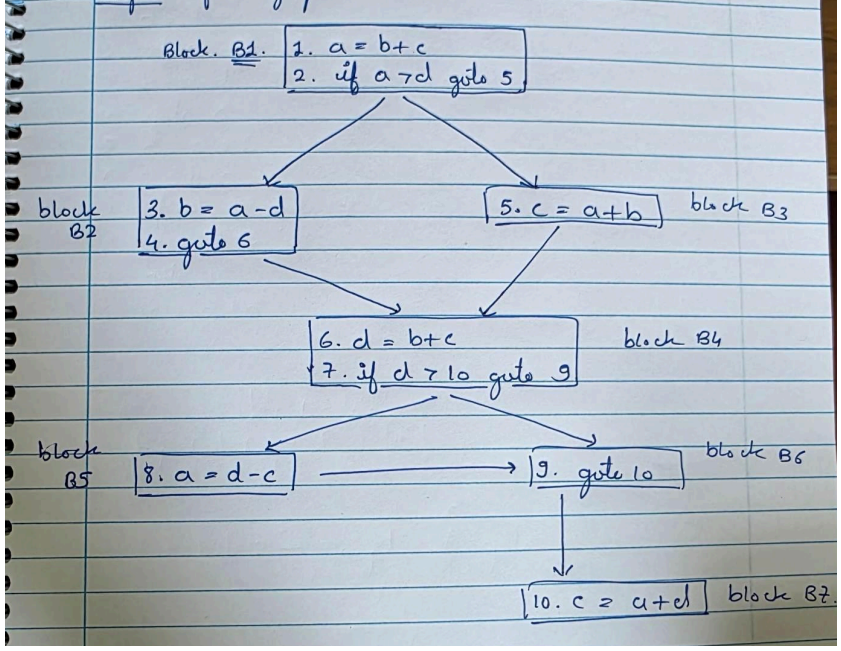
6) Block B6
9. goto 10

7) Block B7
10. c = a + d

∴ Total Basic Blocks = 7.

Date: . . .

Step 3: flow graph.

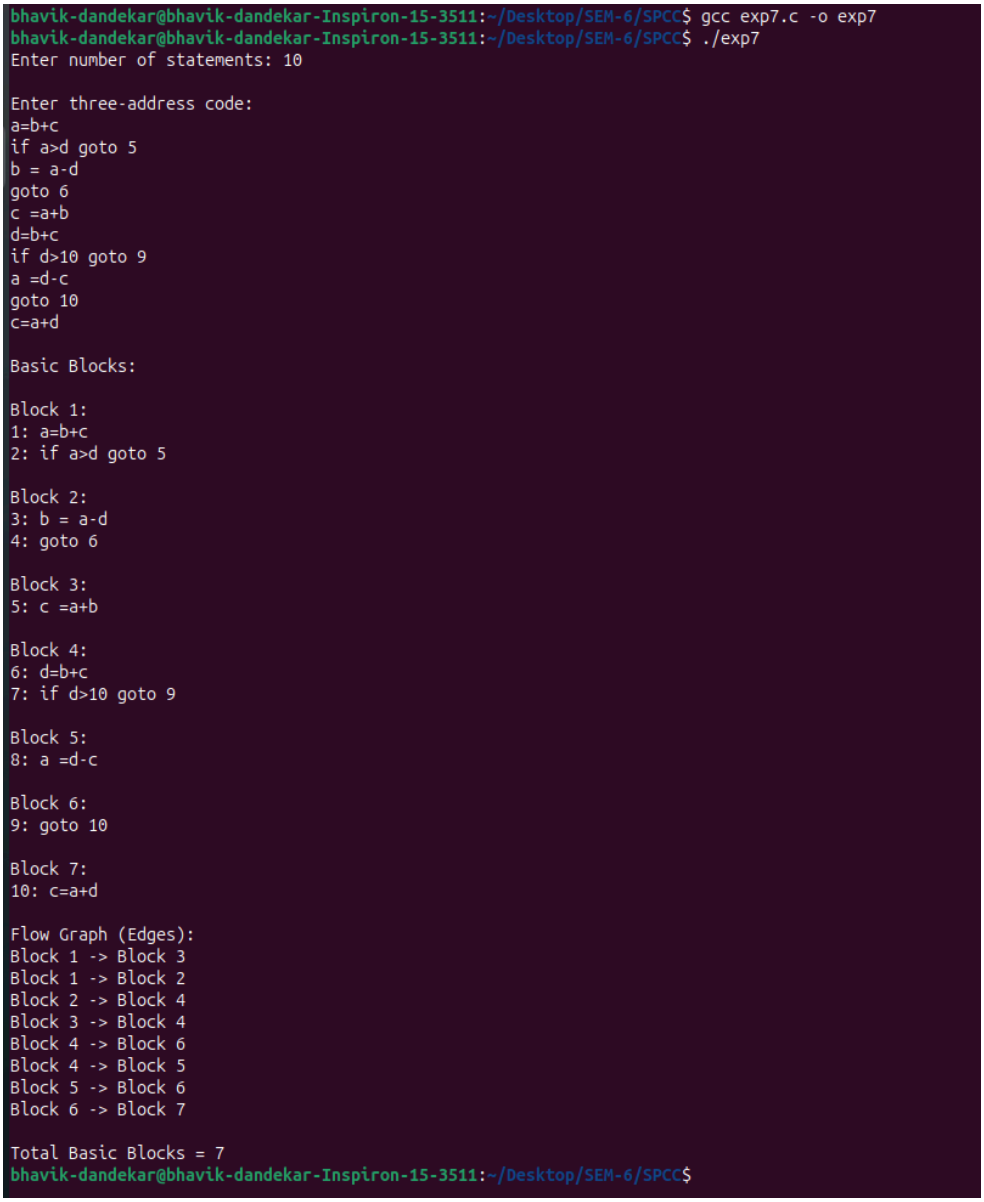


Code	

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 100
#define LEN 100

char code[MAX][LEN];
int leader[MAX];     // marks leaders
int block[MAX];      // block number of each statement
int n;

// Function to check if a line contains "goto"
int isGoto(char *line) {
    return (strstr(line, "goto") != NULL);
}

// Function to extract target line number from goto
int getTarget(char *line) {
    int target = -1;
    char *ptr = strstr(line, "goto");
    if (ptr != NULL) {
        sscanf(ptr, "goto %d", &target);
    }
    return target;
}

int main() {
    printf("Enter number of statements: ");
    scanf("%d", &n);
    getchar(); // consume newline

    printf("\nEnter three-address code:\n");
    for (int i = 0; i < n; i++) {
        fgets(code[i], LEN, stdin);
    }

    // Step 1: Initialize leaders
    for (int i = 0; i < n; i++)
        leader[i] = 0;
```

```
// Rule 1: First statement is leader
leader[0] = 1;

// Rule 2 & 3: Based on goto
for (int i = 0; i < n; i++) {
    if (isGoto(code[i])) {
        int target = getTarget(code[i]);

        if (target > 0 && target <= n)
            leader[target - 1] = 1;

        if (i + 1 < n)
            leader[i + 1] = 1;
    }
}

// Step 2: Assign blocks
int blockNum = 0;
for (int i = 0; i < n; i++) {
    if (leader[i])
        blockNum++;

    block[i] = blockNum;
}

// Output Basic Blocks
printf("\nBasic Blocks:\n");
for (int b = 1; b <= blockNum; b++) {
    printf("\nBlock %d:\n", b);
    for (int i = 0; i < n; i++) {
        if (block[i] == b)
            printf("%d: %s", i + 1, code[i]);
    }
}

// Flow Graph
printf("\nFlow Graph (Edges):\n");

for (int i = 0; i < n; i++) {
```

```
        if (isGoto(code[i])) {
            int target = getTarget(code[i]);
            if (target > 0 && target <= n) {
                printf("Block %d -> Block %d\n",
                        block[i], block[target - 1]);
            }
        }

        // Sequential flow (fall-through)
            if (i + 1 < n) {
                // If NOT unconditional goto
                if (!(isGoto(code[i]) && strstr(code[i], "if") == NULL)) {
                    if (block[i] != block[i + 1]) {
                        printf("Block %d -> Block %d\n",
                                block[i], block[i + 1]);
                    }
                }
            }
    }

    printf("\nTotal Basic Blocks = %d\n", blockNum);

    return 0;
}
```

Snapshot of Output	 bhavik-dandekar@bhavik-dandekar-Inspiron-15-3511:~/Desktop/SEM-6/SPCC$ gcc exp7.c -o exp7 bhavik-dandekar@bhavik-dandekar-Inspiron-15-3511:~/Desktop/SEM-6/SPCC$ ./exp7 Enter number of statements: 10 Enter three-address code: a=b+c if a>d goto 5 b = a-d goto 6 c =a+b d=b+c if d>10 goto 9 a =d-c goto 10 c=a+d Basic Blocks: Block 1: 1: a=b+c 2: if a>d goto 5 Block 2: 3: b = a-d 4: goto 6 Block 3: 5: c =a+b Block 4: 6: d=b+c 7: if d>10 goto 9 Block 5: 8: a =d-c Block 6: 9: goto 10 Block 7: 10: c=a+d Flow Graph (Edges): Block 1 -> Block 3 Block 1 -> Block 2 Block 2 -> Block 4 Block 3 -> Block 4 Block 4 -> Block 6 Block 4 -> Block 5 Block 5 -> Block 6 Block 6 -> Block 7 Total Basic Blocks = 7 bhavik-dandekar@bhavik-dandekar-Inspiron-15-3511:~/Desktop/SEM-6/SPCC$
conclusion	In this experiment, I studied how to divide a sequence of three-address code into basic blocks using leader identification rules. Learned that a basic block represents a straight-line code segment with a single entry and exit point, which simplifies program analysis. Also constructed the flow graph (control flow graph) to represent the flow of control between different basic blocks. This helped in understanding how execution proceeds in the presence of conditional and unconditional jumps. In particular, I observed that conditional statements generate two control paths (target and

	fall-through), whereas unconditional jumps generate only one. Through this experiment, we gained a clear understanding of: ● Identification of leaders ● Formation of basic blocks ● Construction of flow graphs ● Representation of control flow in programs Overall, this experiment provided insight into the internal working of a compiler and how control flow analysis is used as a foundation for code optimization and efficient program execution.